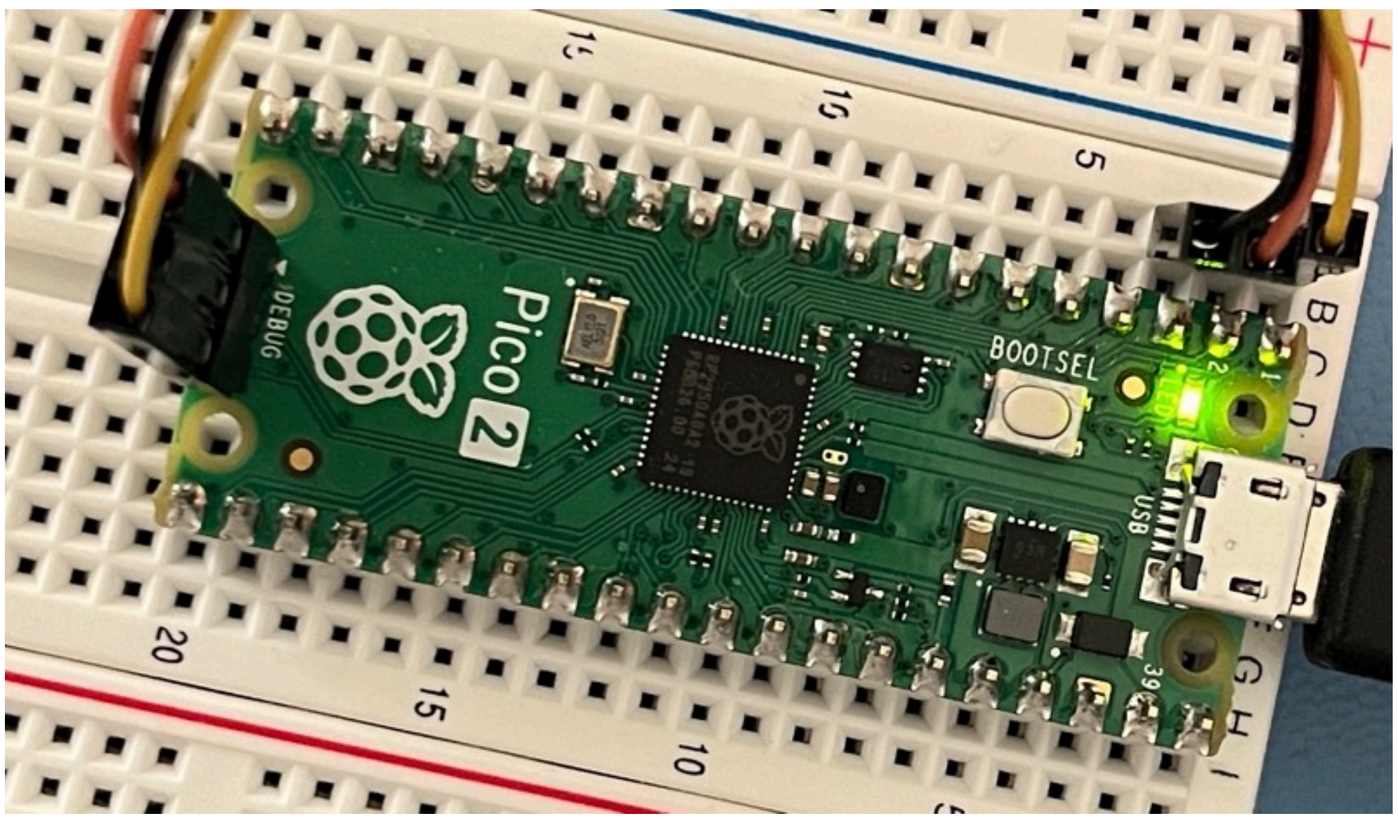
Using a Pico 2 RP2350 without the Pico SDK Build System

September 11, 2024

Raspberry Pi launched Pico 2 featuring the new RP2350 microcontroller last month.

RP2350 is an interesting MCU. It has two cores, and one or both of its cores can be configured as an Arm Cortex-M33 or a Hazard3 RISC-V. For this post I will ignore the RISC-V capability.



Raspberry Pi Pico 2

Pico 2 is not the only board featuring a Cortex-M33 microcontroller, but I believe it is the cheapest and possibly the most easy one to buy around the world. It is around 5 USD. However, from programming perspective, it has one major difference comparing to other Arm Cortex-M33 MCUs such as STM32H5 series. The difference is that RP2350 has a (fixed) bootrom which gives some extra capabilities. The impact of this is that RP2350 first executes its bootrom not the user application. Moreover, I believe in order to support its features (such as versioning, A/B versions, secure boot, signed images) there is a concept of blocks (and partition tables). Partition tables are optional but the use of blocks is a must, so a simple bare-metal application written in C can be compiled using the Arm GNU Toolchain (Pico SDK provides RP2350 CMSIS support) but it cannot be executed directly by the RP2350, because it will not have the required (metadata) blocks describing the image.

In the Pico SDK build system, the required block is defined in `pico-sdk/src/rp2_common/pico_crt0/embedded_start_block.inc.S` and `pico-sdk/src/rp2_common/pico_crt0/embedded_end_block.inc.S`. These are assembly source files with some ifdefs and the resulting data defined in these files is put into a section called `.embedded_block`. Then this block is embedded into the image as defined in the linker description scripts in `pico-sdk/src/rp2_common/pico_crt0/rp2350`.

The minimum block for the metadata of a user application binary is described in RP2350 Datasheet section 5.9.5. Minimum Viable Image Metadata. This metadata is stored in a block (IMAGE_DEF) which contains two items: PICOBIN_BLOCK_ITEM_1BS_IMAGE_TYPE and PICOBIN_BLOCK_ITEM_2BS_LAST. Each block requires an item of type LAST as the last item, so this means this minimum block actually contains only one meaningful item (IMAGE_TYPE). The image type item contains the following information about the image:

- Image is executable (IMAGE_TYPE_EXE)
- Image runs in secure mode (EXE_SECURITY_S)
- Image is for the Arm architecture (EXE_CPU_ARM)
- Image is for RP2350 (EXE_CHIP_RP2350)

This block is 20-bytes long and it should be within the first 4KB of the binary loaded to the flash. That is how the bootroom code finds it.

All (EXE, ARM, RP2350) but one (SECURITY_S) information above should be obvious but you might be asking why it runs in secure mode. The reason is RP2350 security extensions are always enabled (it cannot be disabled like in STM32 MCUs). The bootroom runs in secure mode and makes a normal jump to the user application, hence (at least initially) the user application should run in secure mode. Afterwards, the user application can configure the security if required.

Since it is customary to keep the vector table at the begining of a flash image, and the vector table is less than 4KB, it makes sense to put this 20-byte long block just after the vector table. Similar to the Pico SDK build system, this block can be easily created in an assembly source file and linked to the correct location using the linker description script.

I have created a very simple blinking LED project not using the Pico SDK build system. It can be found at rp2350-bare-metal-build@github.

This work is licensed under a Creative Commons Attribution-NonCommercial-ShareAlike 4.0 International License.